

based search).

- $k=3$ specifies that the algorithm should consider the three nearest neighbours.

Clustering with K-Means

```
import (
    "github.com/sjwhitworth/golearn/clustering"
)

// Create a K-Means model with 3 clusters
kmeans := clustering.NewKMeans(3, 10) // 3 clusters, 10
iterations
kmeans.Fit(trainingData)
clusters := kmeans.Predict(testData)
```

Explanation:

- `NewKMeans(3, 10)` creates a K-Means model with 3 clusters and 10 iterations.
 - `Fit(trainingData)` trains the model on the dataset.
 - `Predict(testData)` assigns test samples to clusters.
- There are several advantages of using Go for ML when compared to Python and R:
- Go runs compiled binaries, making it faster than Python.
 - Go's garbage collection reduces overhead compared to R.
 - Go's goroutines enable more efficient parallel execution than Python's multi-threading.
- Here's an example of running ML tasks concurrently in Go:

```
import "sync"

var wg sync.WaitGroup

wg.Add(2) // Add two concurrent tasks

go func() {
    defer wg.Done()
    tree.Fit(trainingData) // Train decision tree in parallel
}()

go func() {
    defer wg.Done()
    knnClassifier.Fit(trainingData) // Train KNN model in
parallel
}()

wg.Wait() // Wait for both tasks to finish
```

Explanation:

- `sync.WaitGroup` is used to wait for multiple goroutines to complete.
- `go func() { ... }()` runs training tasks in parallel.
- `wg.Done()` marks a task as completed.

Working with GoLearn: A step-by-step guide

Loading and exploring data

Reading datasets with GoLearn: GoLearn provides utilities to load datasets in a structured format using the base package.

```
package main

import (
    "fmt"
    "github.com/sjwhitworth/golearn/base"
)

func main() {
    // Load a CSV dataset into GoLearn's Instances structure
    data, err := base.ParseCSVToInstances("dataset.csv",
true)
    if err != nil {
        fmt.Println("Error loading data:", err)
        return
    }
    // Print dataset summary
    fmt.Println("Dataset Loaded Successfully!")
    fmt.Println(data)
}
```

Explanation:

- `ParseCSVToInstances("dataset.csv", true)` loads the dataset from a CSV file.
- The second argument (`true`) specifies that the CSV contains a header row.
- `fmt.Println(data)` prints a summary of the dataset.

Data exploration techniques in GoLearn

```
import "github.com/sjwhitworth/golearn/evaluation"

// Get feature names and basic statistics
fmt.Println("Number of Features:", data.Cols)
fmt.Println("Number of Rows:", data.Rows)
```

Explanation:

- `Cols` returns the number of columns (features).
- `Rows` returns the number of records (samples).

Data preprocessing

Handling missing values: GoLearn doesn't have built-in missing value handling, but you can manually clean your dataset.

```
import "github.com/sjwhitworth/golearn/base"

// Iterate over dataset and remove rows with missing values
filteredData := base.NewInstances()
```